

Общая часть

Перед началом курса

Убедитесь, что у Вас есть полноценный доступ к физтех почте и гитлаб аккаунту.

1. [Заполнить форму регистрации](#) - обязательно всем
2. Присоединиться к [чату курса](#) и [каналу](#) с новостями- обязательно всем
3. Если возникли проблемы, то заполни [форму](#)
4. [Страница ВИКИ курса](#)
5. ГИТЛАБ - <https://gitlab.atp-fivt.org/courses-public/db-supplementary>

Установка ПО

Вводная лекция о настройке ПО и необходимых приложениях

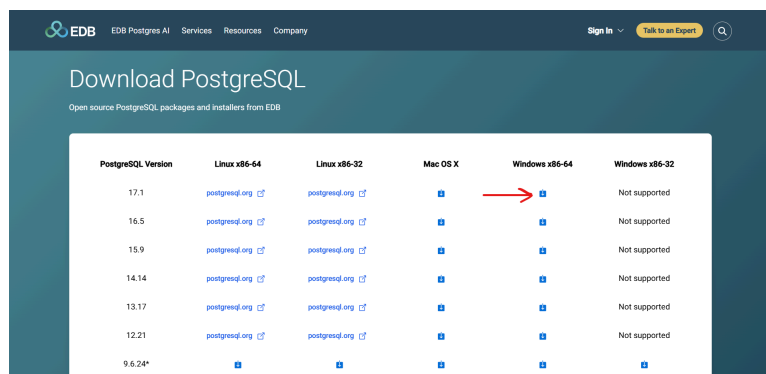
Windows

В качестве интерфейсного приложения для СУБД будем использовать DBeaver - один из самых простых способов поднять БД и писать SQL-запросы.

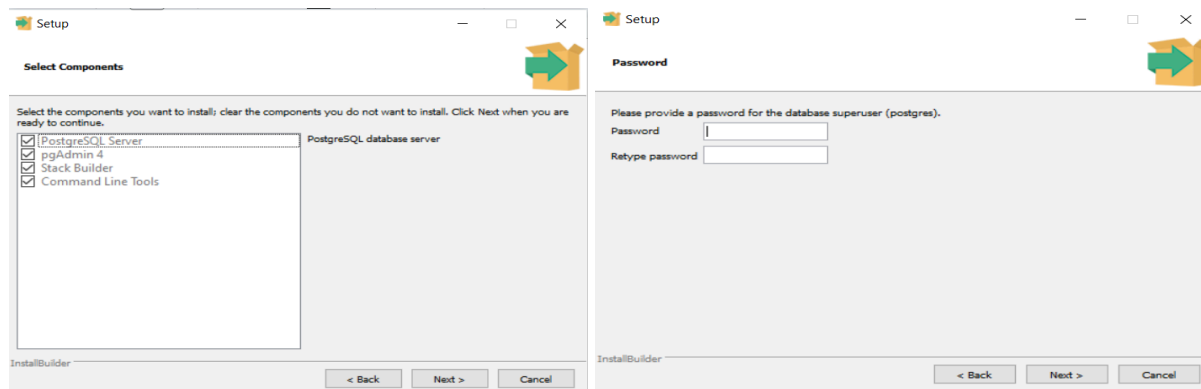
1. Установка PostgreSQL + pgAdmin

Устанавливаем последнюю версию PostgreSQL для Windows по ссылке:

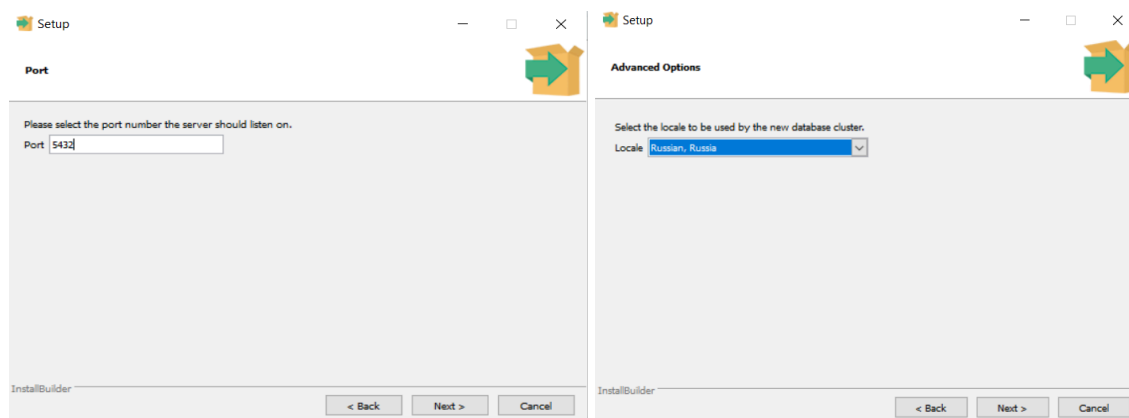
<https://www.enterprisedb.com/downloads/postgres-postgresql-downloads>



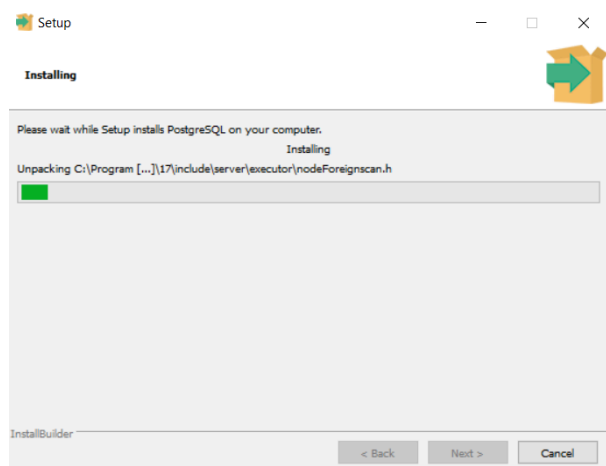
Выбираем все компоненты: PostgreSQL Server, pgAdmin4, Stack Builder, Command Line Tools. Придумываем пароль для суперюзера (логином будет *postgres* - это мы потом еще увидим, когда зайдем в pgAdmin):



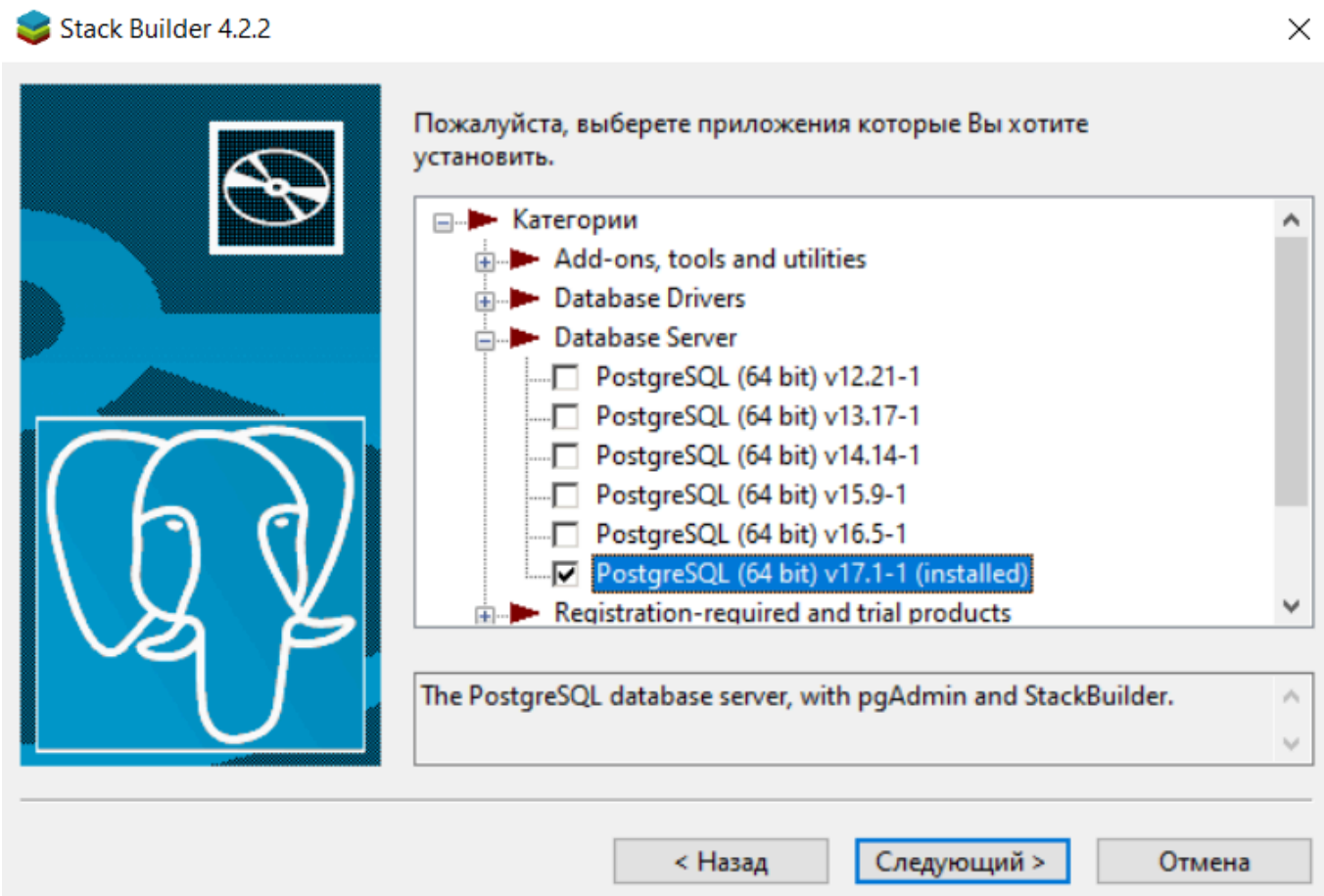
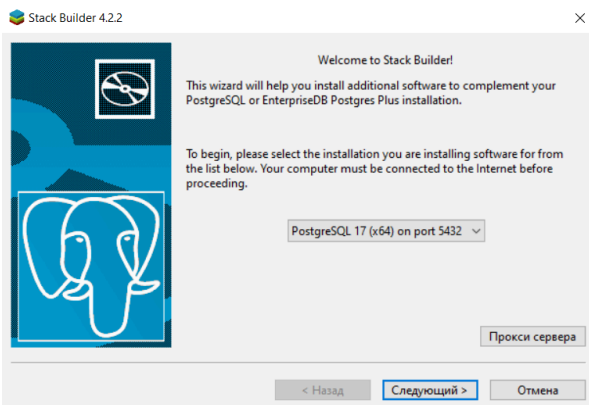
Оставляем дефолтный порт 5432 для PostgreSQL (если стоит другой - лучше поставить 5432), выбираем Россию как нашу локацию:



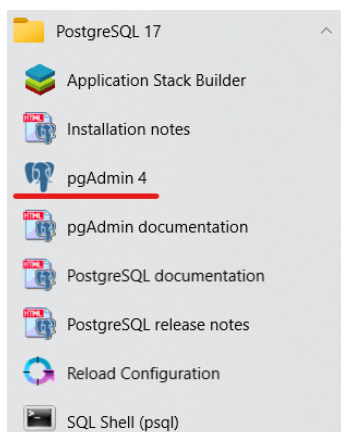
Дальше протыкиваем и начинаем установку:



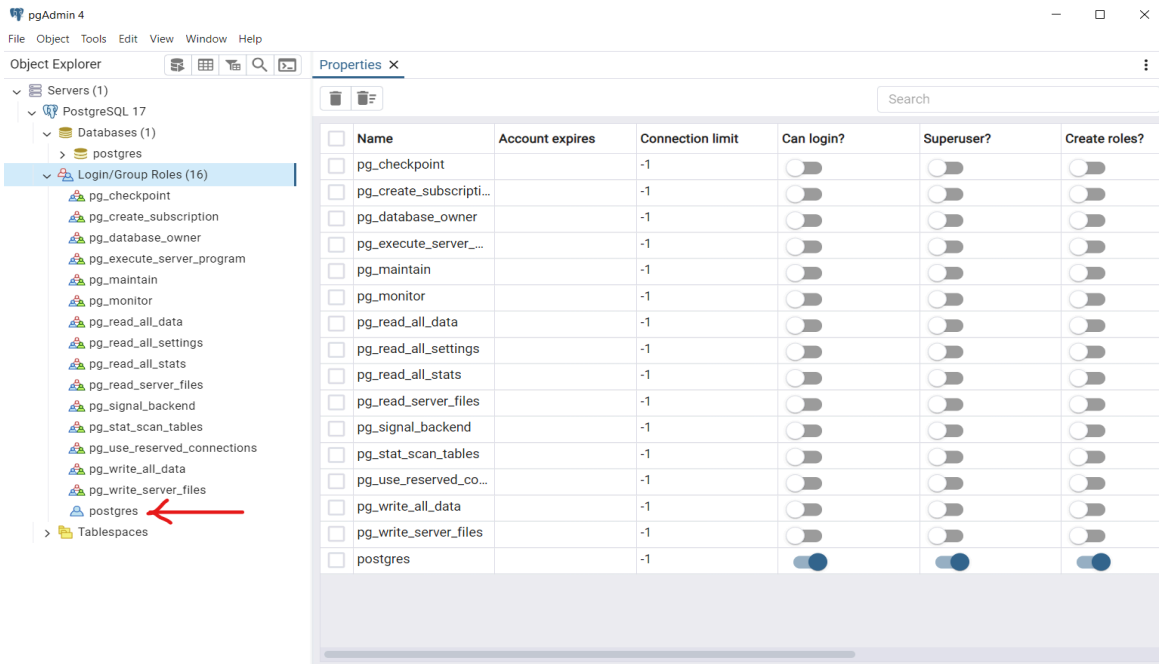
PostgreSQL с дополнительными компонентами установлен. Дальше выбираем параметры для установки Stack Builder как на скринах:



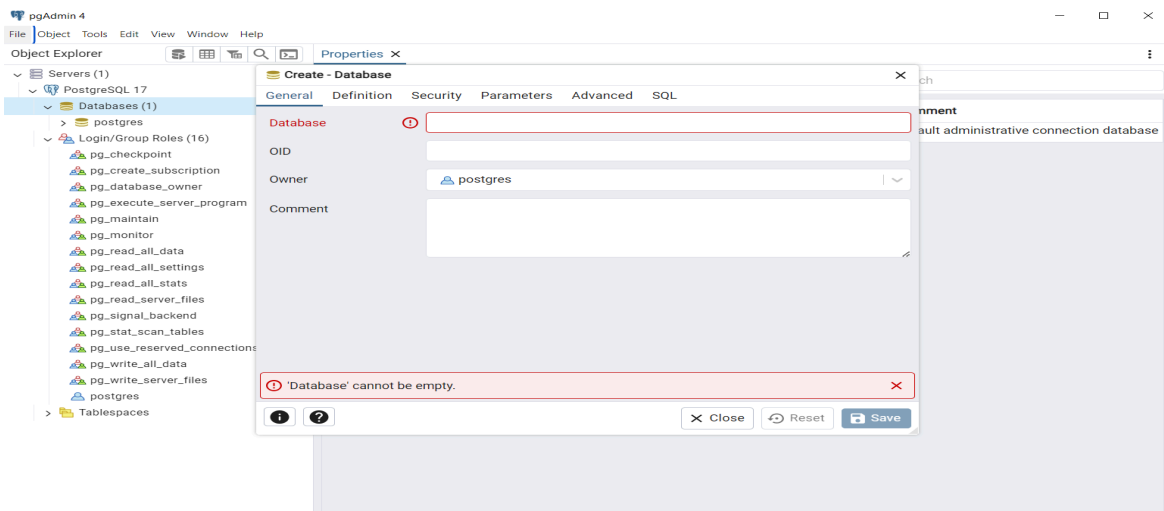
Завершаем установку. Идем в pgAdmin:



В интерфейсе все хорошо - видим нашего суперюзера:



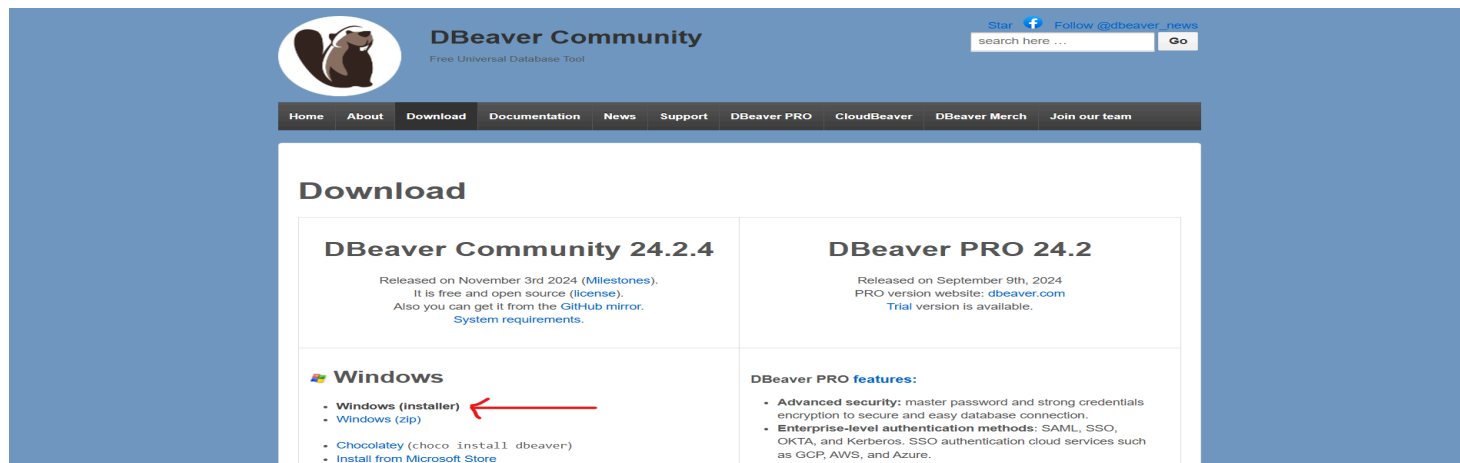
Сейчас создана дефолтная база данных postgres. Ткнув на Databases правой кнопкой мыши, можно создать новую базу данных. Будет доступен следующий интерфейс:



При желании можно создать новую (достаточно указать только имя, остальные поля можно не трогать). Я в гайде этого делать не буду.

2. Установка DBeaver

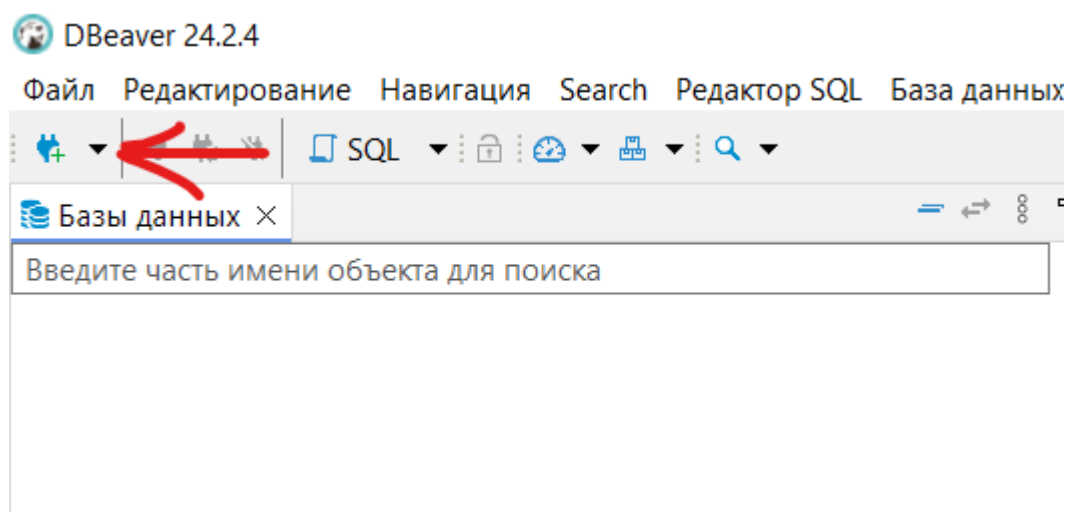
Переходим на <https://dbeaver.io/download/> и делаем тук на Windows (installer)



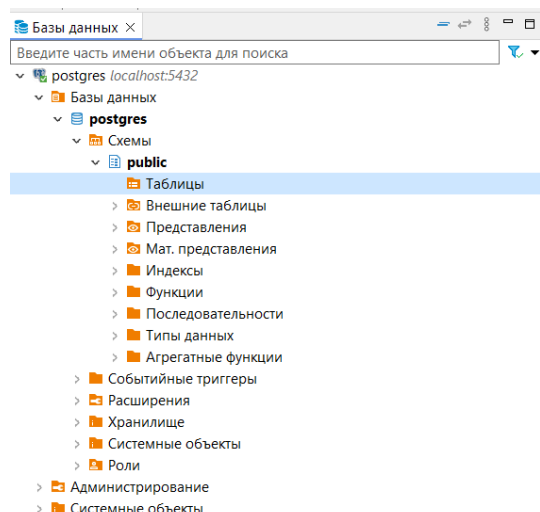
При установке выбираем Include Java, остальное в большинстве случаев не нужно. На остальных вкладках ничего можно ничего не менять. Desktop Shortcut по желанию, я себе создал.

3. Работа с интерфейсом

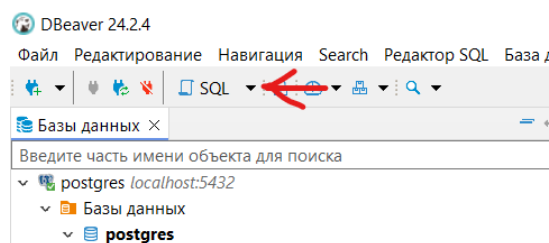
Нажав на элемент на скрине, создадим подключение к PostgreSQL:



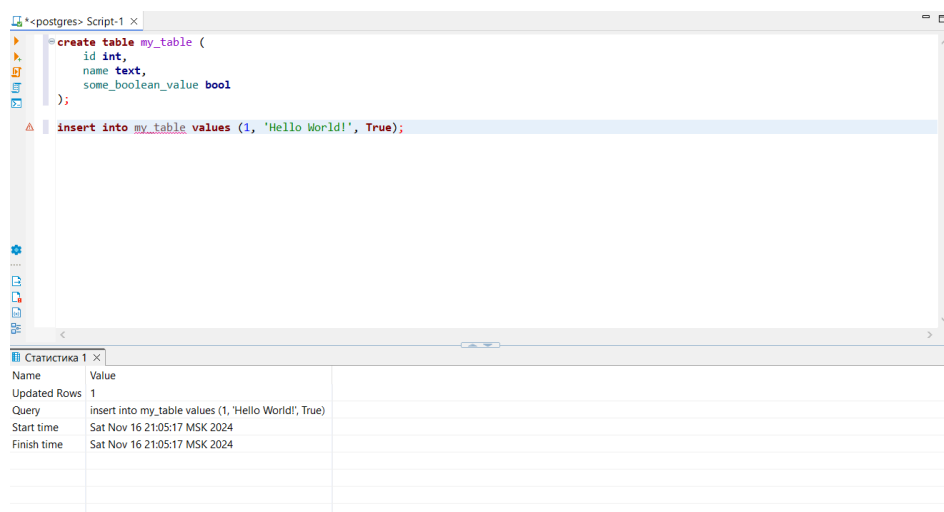
В появившемся окне вводим пароль от суперюзера и делаем проверку. Выполняем тест соединения. Если произошла ошибка, скорее всего что-то было сделано не так. Создаем. Должно получиться что-то такое:



Пока здесь таблиц нет. Чтобы создать их (и в целом написать любой скрипт), нужно нажать на следующий элемент:



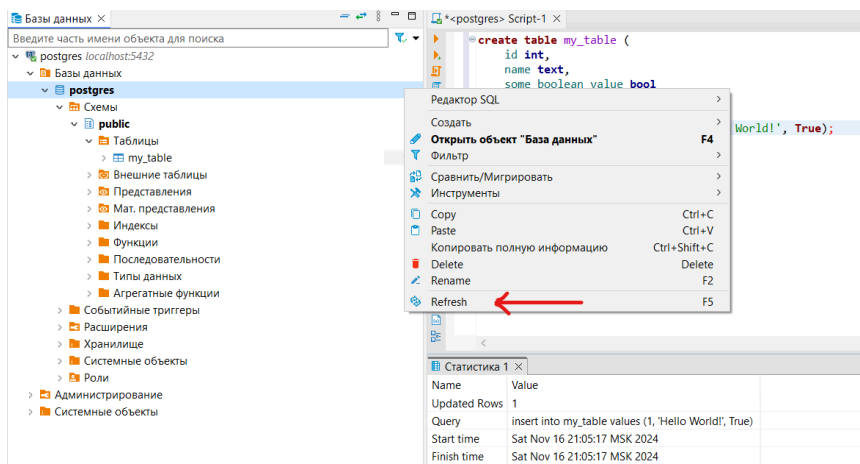
Пишем скрипт. Например, для начала работы, можно выполнить такой скрипт. Чтобы выполнить, нужно нажать “выполнить скрипт” - 3 кнопка по счету (альтернативно: можно выделить весь скрипт и нажать на треугольник слева-сверху относительно скрина). Чтобы выполнить только части скрипта, нужно выделить нужные строки. Пример скрипта:



Кроме того, можно использовать горячие клавиши Ctrl + Enter для запуска выделенной части скрипта и Alt + X для выполнения всего скрипта.

Запросы также можно выполнять в sql-консоли (находится там же, где и создание sql-редактора - 2 скрина выше). Обычно он предназначен для маленьких запросов, рекомендую пользоваться именно редактором.

После refresh-a postgres (скрин ниже) изменения в схему подтянутся сами:



Вводная часть для Windows на этом закончилась. Дальше можно найти видео в Youtube для создания других элементов БД (представлений, функций, индексов, ...).

MacOS

PostgreSQL на MacOS

Установка через графический интерфейс:

1. Загрузка установочного файла:

- Перейдите на официальный сайт PostgreSQL [[ссылка](#)].
- Выберите версию для macOS и скачайте установочный пакет (обычно используется пакет "Postgres.app" или "EnterpriseDB").

2. Установка пакета:

- Откройте загруженный установочный файл и следуйте инструкциям установщика.
- Установите путь для PostgreSQL, добавив его в системные переменные, если это необходимо (обычно установщик делает это автоматически).

В этот установщик входит:

Сервер PostgreSQL

pgAdmin, графический инструмент для управления и разработки ваших баз данных

StackBuilder, менеджер пакетов для загрузки и установки дополнительных инструментов и драйверов PostgreSQL. Stackbuilder включает в себя управление, интеграцию, миграцию, репликацию, геопространственные данные, коннекторы и другие инструменты.

Этот установщик может работать в графическом режиме, режиме командной строки или в режиме тихой установки.

Продвинутые пользователи также могут загрузить [zip-архив](#) бинарных файлов без установщика. Эта загрузка предназначена для пользователей, которые хотят включить PostgreSQL как часть другого установщика приложения.

3. По аналогии с Windows проверяем работоспособность установленных файлов.

Установка через командную строку:

1. Установка Homebrew:

- Убедитесь, что Homebrew установлен. Введите команду в терминале:

```
brew --version
```

Если Homebrew не установлен, выполните:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Установка PostgreSQL:

Теперь, когда у нас установлен Homebrew - мы можем установить PostgreSQL прямо через командную строку.

```
brew install postgresql
```

В случае необходимости в установке конкретной версии, например, для установки PostgreSQL 15 можно использовать следующую команду:

```
brew install postgresql@15
```

Как проверить, что заработало?

1. Проверьте версию PostgreSQL: Откройте терминал и введите:

```
psql --version
```

Note: Если возникла ошибка:


```
zsh: command not found: psql
```

Решение:

- Добавьте путь к PostgreSQL в PATH
 - Узнайте, где именно Homebrew установил PostgreSQL. Обычно путь будет следующим (На примере 15-ой версии):

```
/opt/homebrew/opt/postgresql@15/bin
```

- Тут может возникнуть другая ошибка:

```
zsh: permission denied: /opt/homebrew/opt/postgresql@15/bin
```

Возможные решения:

- а. Проверьте правильность команды: Убедитесь, что вы не пытаетесь запустить саму директорию `/opt/homebrew/opt/postgresql@15/bin` как команду. Используйте полные команды, например:

```
/opt/homebrew/opt/postgresql@15/bin/psql --version
```

- б. Права доступа: Если проблема связана с правами доступа, убедитесь, что файлы внутри этой директории имеют исполняемые права:

```
ls -l /opt/homebrew/opt/postgresql@15/bin
```

Если вы видите, что файлы не имеют исполняемых прав (например, отсутствуют `x` в правах), исправьте это:

```
chmod +x /opt/homebrew/opt/postgresql@15/bin/*
```

- в. Проверка установки Homebrew: Убедитесь, что PostgreSQL установлен корректно через Homebrew:

```
brew doctor  
brew info postgresql@15
```

- д. Использование правильного пути: Проверьте, что вы правильно добавили путь к PATH в `.zshrc`:

```
export PATH="/opt/homebrew/opt/postgresql@15/bin:$PATH"
```

Затем выполните:

```
source ~/.zshrc
```

Попробуйте после этих шагов снова выполнить команду:

```
psql --version.
```

Теперь, продолжим проверку:

2. Запустите службу PostgreSQL: Убедитесь, что PostgreSQL запущен. Выполните команду:

```
brew services start postgresql@15
```

Если служба уже запущена, можно перезапустить:

```
brew services restart postgresql@15
```

Проверьте статус службы: Убедитесь, что PostgreSQL работает:

```
brew services list
```

В этом списке найдите строку с `postgresql@15` и убедитесь, что статус – `started`.

Подключитесь к PostgreSQL: Попробуйте подключиться к серверу с помощью:

```
psql postgres
```

Если подключение успешно, введите:

```
SELECT version();
```

Это покажет информацию о версии PostgreSQL и подтвердит, что сервер работает корректно!

Ах, да, главное - как выйти?) В терминал ввести: `\q`

Проверка и управление через командную строку:

1. Создание базы данных:

```
createdb mydb
```

2. Подключение к базе данных:

```
psql mydb
```

3. Создание таблиц и работа с данными: Введите SQL-команды для создания таблиц и выполнения запросов, например:

```
CREATE TABLE test_table (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(100)  
);  
INSERT INTO test_table (name) VALUES ('Example');  
SELECT * FROM test_table;
```

Дополнительные ресурсы по данной вкладке:

1. Документация PostgreSQL [[ссылка](#)]
2. Сайт Homebrew [[ссылка](#)]
3. Полезные ссылки на youtube с гайдами по установке:
 - а. На английском: [[ссылка](#)] [[ссылка](#)]
 - б. На русском: [[ссылка](#)]
 - с. Homebrew: [[ссылка](#)]
4. Для новичков, работа с терминалом - статья на хабре [[ссылка](#)]

DBeaver на MacOS

Установка DBeaver через графический интерфейс (GUI):

1. Скачивание установочного файла:
 - Перейдите на официальный сайт DBeaver: <https://dbeaver.io/download/>.
 - Выберите версию для macOS и скачайте установочный файл (обычно формат .dmg).
2. Установка DBeaver:
 - Откройте загруженный файл .dmg и перетащите иконку DBeaver в папку "Applications".
 - После копирования приложения перейдите в "Applications" и запустите DBeaver.
3. Настройка подключения к базе данных:
 - Запустите DBeaver и выберите "New Database Connection".
 - Выберите нужный драйвер (например, PostgreSQL).
 - Введите данные подключения: "Host", "Port" (по умолчанию 5432), "Database", "Username" и "Password".
 - Нажмите "Test Connection", чтобы убедиться, что соединение установлено.

4. Работа с базой данных:

- После успешного подключения откройте панель "SQL Editor" и начните выполнение запросов.

! Более подробную инструкцию и работу с DBeaver вы можете найти в гайде для Windows.

Установка DBeaver через командную строку:

1. Установка Homebrew: Если у вас еще не установлен Homebrew, выполните:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Установка DBeaver:

Выполните команду для установки DBeaver Community Edition:

```
brew install --cask dbeaver-community
```

3. Запуск DBeaver:

После установки запустите DBeaver из папки “Applications” или с помощью команды:

```
open -a DBeaver
```

4. Настройка подключения к базе данных:

Следуйте шагам, описанным в Часть 1, начиная с шага 3, чтобы создать новое подключение.

Проверка работы и базовая навигация:

1. Создание и выполнение SQL-запросов:

В “SQL Editor” введите запрос и нажмите “Run” (или используйте сочетание клавиш Cmd + Enter).

2. Навигация по базе данных:

Используйте панель “Database Navigator” для поиска таблиц, представлений и других объектов.

Дополнительные советы и ссылки:

- Настройка темной темы: В меню “Preferences” выберите “Appearance” и переключите тему на “Dark” для комфортной работы.
- Установка плагинов: DBeaver поддерживает дополнительные плагины, которые можно установить через “Help → Install New Software”.

Ссылки на полезные tutorиалы:

1. Общий обзор: [[ссылка](#)]
2. Обучение: [[ссылка](#)]

DataGrip на MacOS

Установка DataGrip через графический интерфейс:

1. Скачивание установочного файла:
 - Перейдите на официальный сайт JetBrains: [[ссылка](#)].
 - Нажмите на кнопку "Download" и выберите версию для macOS.
2. Установка DataGrip:
 - Откройте загруженный файл .dmg и перетащите иконку DataGrip в папку "Applications".
 - Перейдите в "Applications" и запустите DataGrip.
3. Настройка подключения к базе данных:
 - Нажмите на "File → Data Sources and Drivers".
 - Выберите тип базы данных (например, PostgreSQL, MySQL и др.).
 - Введите параметры подключения: "Host", "Port", "Database", "Username" и "Password".
 - Нажмите "Test Connection" для проверки подключения.
4. Работа с базой данных:
 - После успешного подключения откройте "Database Explorer" и начните выполнение запросов через редактор SQL.

Установка DataGrip через командную строку:

По аналогии с DBeaver можно установить через Homebrew:

```
brew install --cask datagrip
```

Дополнительные советы и ссылки:

- Настройка внешнего вида: Перейдите в "Preferences → Appearance & Behavior" для выбора светлой или темной темы.

- Интеграция с VCS: DataGrip поддерживает работу с системами контроля версий, такими как Git.
- Горячие клавиши: Ознакомьтесь с быстрыми командами в разделе "Help → Keyboard Reference".
- Обзор от Jetbains: [[ссылка](#)]

Linux

Установка на хост машину

Для Ubuntu/Debian:

Unset

```
# Обновите список пакетов
sudo apt update
```

```
# Установите PostgreSQL
sudo apt install postgresql postgresql-contrib -y
```

```
# Проверьте статус службы PostgreSQL
sudo systemctl status postgresql
```

Для CentOS/RHEL:

Unset

```
# Установите репозиторий PostgreSQL
sudo dnf install -y https://download.postgresql.org/pub/repos/yum/reporpms/EL-$(rpm
-E %{rhel})-x86_64/pgdg-redhat-repo-latest.noarch.rpm
```

```
# Установите PostgreSQL
sudo dnf install -y postgresql15-server postgresql15
```

```
# Инициализируйте базу данных и запустите службу
sudo /usr/pgsql-15/bin/postgresql-15-setup initdb
sudo systemctl enable postgresql-15
sudo systemctl start postgresql-15
```

Установка через Docker контейнер

Prerequisites:

- [Docker](#). Можно установить с официального сайта. Не рекомендуется устанавливать Docker Desktop на linux.
- [Docker compose](#). Устанавливается одной командой через пакетный менеджер.

Далее создайте в рабочей директории файл `docker-compose.yml` и поместите в него следующий контент:

```
Unset
version: "3.8"

services:
  db:
    image: postgres:15-alpine
    container_name: "postgres_db"
    environment:
      - POSTGRES_USER=postgres
      - POSTGRES_PASSWORD=postgres
      - POSTGRES_DB=pg_db
      - POSTGRES_PORT=5432
      - PGDATA=/var/lib/postgresql/data/pgdata
    volumes:
      - postgres_data:/var/lib/postgresql/data/pgdata
    ports:
      - "5432:5432"
    restart: always
#   env_file: # для хранения secrets правильнее использовать .env-файл
#   - .env
  pgadmin:
    image: dpage/pgadmin4
    container_name: "pgadmin"
    environment:
      - PGADMIN_DEFAULT_EMAIL=admin@admin.com
      - PGADMIN_DEFAULT_PASSWORD=root
    volumes:
      - data_pgadmin:/var/lib/pgadmin
    ports:
      - "8080:80"

volumes:
  postgres_data:
  data_pgadmin:
```

Для работы с контейнером используйте следующие команды:

```
Unset
# Из директории с docker-compose.yml файлом
docker-compose up [-d] (если в режиме detached)
docker-compose down [-v] (чтобы удалить контейнер)
```

Так же вот [список](#) команд по работе с docker

Работа с PostgreSQL через командную строку

По умолчанию на unix-системах при установке Postgres создаётся пользователь с именем postgres и паролем postgres

Как войти в консоль Postgres:

Unset

```
# зайти под соответствующим пользователем
sudo -i -u postgres

# Запустить консоль
psql

# Запустить консоль с явным указанием параметров
psql -U myuser -d mydb -h 127.0.0.1 -p 5432
```

Основные команды psql:

Unset

```
-- Показать базы данных
\l

-- Подключиться к базе данных
\c mydb

-- Показать таблицы
\dt

-- Выйти из консоли
\q

-- Создать таблицу
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100) UNIQUE NOT NULL
);

-- Вставить данные
INSERT INTO users (name, email) VALUES ('Alice', 'alice@example.com');

-- Получить данные
SELECT * FROM users;

-- Удалить таблицу
DROP TABLE users;
```

Установка клиентов для работы с БД

- [DbVisualizer](#). Малоизвестный, но достаточно удобный.
- [DBeaver](#). Известный, в целом удобный. Не очень красивый UI.
- [DataGrip](#). Самый известный. Очень удобный, но требует лицензию.
- В большинстве продуктов JetBrains есть поддержка работы с БД.

Подключение к БД аналогично инструкции для Windows

P.s. Полезные ссылки:

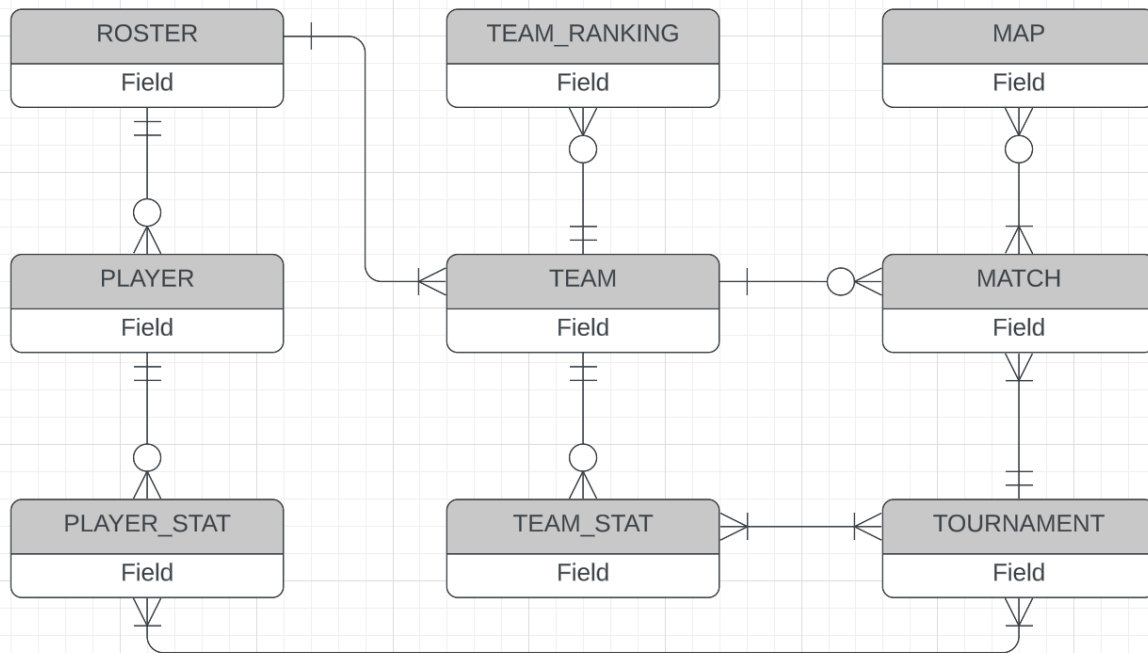
- [Работа с psql](#)
- [Сайт для генерации тестовых данных для проекта](#)

Дополнительные ссылки на приложения для работы с проектом:

[Disclaimer] На нашем курсе при выполнении проекта, потребуются инструменты для визуализации. Существует очень много различных сервисов. Мы разберем самые популярные.

draw.io:

draw.io – бесплатный и простой в использовании инструмент для создания диаграмм.



Выше демонстрация визуализации концептуальной модели БД.

Применение в проектах:

- Создание ER-диаграмм для проектирования базы данных.
- Схемы процессов ETL (Extract, Transform, Load).
- Диаграммы потоков данных для визуализации передачи информации между системами.

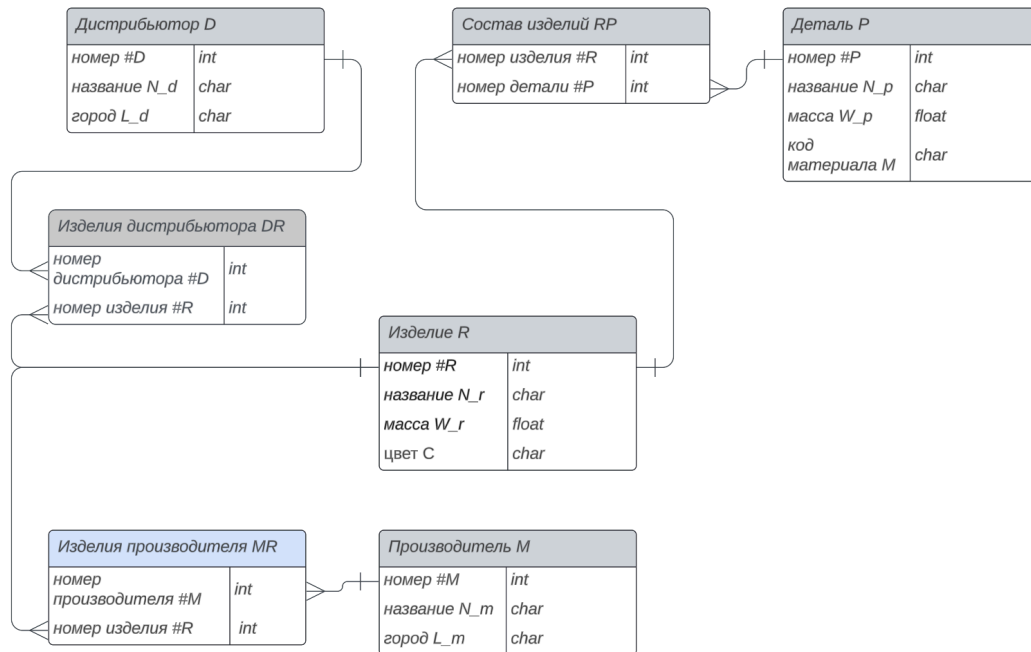
Преимущества:

1. Поддержка работы как онлайн, так и через настольное приложение.
2. Интеграция с облачными хранилищами (Google Drive, OneDrive).

Lucidchart:

Lucidchart - Веб-инструмент для создания сложных диаграмм с поддержкой командной работы. (Весьма похожий аналог draw.io, но с более приятным интерфейсом)

Как видно по картинке - итог с draw.io по визуалу весьма похож, тут уже дело



вкуса.

3. draw.io

4. sql.toad.cz

Контрольные мероприятия

Проект

Общая информация

Проект нужно сдавать по итерациям своему семинаристу(ке) или ассистенту. Работа над проектом **исключительно** индивидуальная.

Тема проекта произвольная, но ее необходимо **утвердить (согласовать)** с семинаристом/ассистентом.

На последнем семинаре (до зачета) защитить проект с презентацией. Все баллы за проект выставляются "финально" только после защиты.

Любые ошибки или неточности найденные в процессе защиты - могут снизить или обнулить заработанные баллы. При защите проекта могут задавать уточняющие вопросы, на которые нужно ответить. Ответы на эти вопросы тоже влияют на финальную оценку проекта.

Защита проект

На последнем (или по договоренности с семинаристами) семинаре проходит защита проекта.

На защите проекта присутствует семинарист, ассистенты + сотрудники курса, которые закреплены за данной группой.

Проверяющим:

1. Составить список/очередь выступления и тайминг
2. Собрать в 1-ой таблице ссылки на гитлаб и документацию к проекту, презентации (все материалы должны быть предоставлены студентами не позже чем за 24 часа до дня защиты)
3. Оценить выступление студента
4. Задать доп вопросы (рекомендуемо от 3шт) и оценить ответы на вопросы (вопросы могут быть из программы курса, связанные с этапами реализации проектов)
5. Поставить финальную оценку (подтвердить оценку) за проект

Студенту:

1. Необходимо продемонстрировать, что базовая часть проекта была защищена в срок.
2. Обозначить кол-во дополнительных итераций, которые были защищены ко времени финальной защиты проекта
3. Продемонстрировать все защищенные пункты на гитлаб
4. Подготовить презентацию о своем проекте (рекомендуемые составные части: введение, архитектура, реализация, демонстрация работоспособности проекта (запуск скриптов), сложности при реализации и возможные улучшения, заключение)
5. Быть готовым “запустить” реализованную часть проекта у себя на локальной машине.

Иметь с собой:

флешку, ноутбук, документ удостоверяющий личность, переходники (если необходимо)

Регламент (рекомендуемый):

1. Подготовка к выступлению до 1 минуты
2. Выступление студента 3-5 минут
3. Дополнительные вопросы и ответы на них (от 3 шт) 3-7 мин

Сдача проекта по итерациям:

Итерация может включать в себя только отправку материалов в гитлаб (и возможном дублировании на почту семинаристу/ассистенту). В таком случае на проверку закладывается до 5 раб.дней. Пожалуйста указывайте в assignee своих ассистентов и/или семинариста (точнее объявит семинарист внутри каждой группы).

Итерация (большинство) сдаются лично ассистенту или семинаристу. Сдача итерации может также включать в себя дополнительным вопросы и задачи для практики, не более 5 шт.

На каждую защиту итерации необходимо приносить в цифровом (и/или печатном виде по требованию) материалы всех предыдущих и текущей итераций. На печатных материалах должна быть проставлена подпись принимающего, предварительная оценка, дата, фио, комментарий (если потребуется).

Итерация считается сданной, только при ответе семинариста/ассистента, что больше никаких исправлений не требуется, и выставления предварительной оценки за итерацию.

Пересдать итерацию после получения предварительной оценки нельзя. Пересдача итерации происходит **только** в случае нахождения **критических** замечаний, требующие исправления перед началом работы над следующей итерацией.

Каждая “сдача” после 1-ой несет за собой снижение максимального возможного для получения балла за итерацию на 30% от текущего максимального балла.

Пример

Номер попытки	Макс возможное кол-во баллов
1	5
2 (1-ая пересдача)	3,5
3 (2-ая пересдача)	2,45

От вас ожидается примерная структура проекта в вашем личном репозитории

```

├── .gitlab-ci.yml      ← описание пайплайна для проекта (по шагам)
├── README.md          ← описание вашего проекта (какая предметная
область и тд)
├── analysis            ← описание
│   ├── __init__.py
│   ├── analysis.py
│   ├── fill_tables.py
│   └── ...
├── docs                ← картинки и таблички
│   ├── conceptual-model.png
│   ├── logical-model.png
│   └── physical-model.md
├── requirements.txt    ← зависимости для Python
├── scripts              ← sql-скрипты (структура этой директории может
быть другой на ваше усмотрение)
│   ├── inserts1.sql
│   ├── ...
│   └── table1_ddl.sql
│   └── ...
├── tests                ← директория с тестами (структура этой директории
может быть другой на ваше усмотрение)
│   ├── __init__.py
│   ├── conftest.py
│   ├── test1.py
│   ├── test2.py
│   └── ...
└── ...                ← любые другие необходимые файлы

```

Также должен быть заполнен readme , хотя бы как здесь:

📁 Проект. Примеры

В README.md файле должны быть разделы:

0 проекте

общее описание проекта

Физ. модель

“картинка” физ модели и комментарии к ней

Сценарии использования

Как запускать, и как проверять Ваш проект.

Скрипты

Описание очередности запуска скриптов, ожидаемых результатов и

ограничений

Где его можно использовать

Обязательно:

После получения предварительной оценки за итерацию (и после защиты), необходимо загрузить материалы данной итерации на гитлаб (с указанием assignee).

В комментариях надо указать :

1. Группу
2. Итерацию
3. Поставленную оценку
4. Комментарии семинариста/ассистента (если есть), который это принял.

Базовая (обязательная) часть

Стадии № 1-7 и пункты внутри них являются **обязательными (необходимыми)**. Без выполнения этих пунктов нельзя получить положительную оценку за курс БД (получить зачет). Оценка Зачет/Незачет.

Для получения “Зачета” по итерациям № 1-7, необходимо каждую итерацию сдавать вовремя и получить оценку “зачтено” до дедлайна, то есть должны быть устранены все критические замечания.

Все эти пункты должны быть выложены на гитлаб, а на всех распечатанных материалах должна стоять подпись ассистента или семинариста.

При сдаче/защите каждого пункта студент должен быть способен показать работающие скрипты, должен быть в состоянии вносить в них изменения, и отвечать на уточняющие, дополнительные вопросы.

1) Защита темы.

- а) Тема может быть абсолютно любой, но при этом все данные и функционал над ними должен быть логичен и оправдан в рамках этой темы.

- b) Также темы не должны повторяться в рамках одной группы (или 2х групп, которые занимаются вместе).

2) Создание концептуальной модели.

- a) Концептуальная модель схемы представляет из себя диаграмму, состоящую из значащих таблиц и связей между ними.
- b) Нормализация на данном этапе не требуется.
- c) Необходимое условие - 5 и более значащих таблиц.

Пример -  Концептуальная модель.png

3) Создание логической модели.

- a) Необходимое условие - 5 и более значащих таблиц.
- b) Подразумевает перевод схемы в одну из нормальных форм (2 или 3, с оправданием выбора),
- c) Наличие минимум одной таблицы с версионными данными (с оправданием типа версионирования SCD 2, SCD 3 или SCD 4).

Пример -  Логическая модель.png

4) Создание физической модели.

- a) Подразумевает определение всей атрибутов всех таблиц схемы с указанием типа данных и всех ограничений.

Пример -  Физическая модель.png

5) Реализация схемы посредством DDL-скриптов.

6) Заполнение схемы данными, с помощью DML-скриптов или прочтения CSV или эксель.

- a) Требования - не менее 15 строк в каждую значащую таблицу.
- b) Не менее 30 строк в каждую таблицу-связку (таблицу, созданную при нормализации схемы, и содержащую ссылки на несколько других, обычно значимых, таблиц).
- c) Не менее 30 строк в каждую таблицу, хранящую версионные данные.

7) Написание не менее 10 осмысленных запросов к схеме, со смысловым описанием желаемого вывода, с использованием:

- a) WHERE,
- b) GROUP BY,
- c) HAVING,
- d) ORDER BY,
- e) JOIN (left, right, full, inner)
- f) подзапросы (скалярные и не скалярные, простые, сложные, с оператором in, any, all, exists, самосоединением)
- g) оконные функции (ранжирующие, агрегирующие, смещающие, математические)
- h) LIMIT/OFFSET

В одном запросе может быть использовано сразу несколько операторов

Дополнительные стадии проекта

1. Создание представлений
 - а. Не менее 2-ух
2. Создание индексов для технических таблиц
 - а. Не менее 2-ух
3. Создание хранимых процедур или функций
 - а. Не менее 3 шт
 - i. Не менее 1 процедуры
 - ii. Не менее 1 функции
4. Создание триггеров
 - а. Не менее 3шт
5. Создание тестов (при помощи pytest) для запросов из пункта №7 (Или других библиотек (ЯП) при предварительном согласовании)
 - а. Не менее 10 тестов + пояснения к ним
 - б. Формирование отчета о “ %” покрытия тестами Вашей работы
6. Используя согласованный язык программирования и библиотеку:
 - а. Сгенерировать данные и с их помощью вставить данные в уже оформленную БД,
 - б. Теми же инструментами извлечь данные (из таблицы на выбор), возможно, предварительно агрегированные средствами СУБД,
 - с. Провести анализ:
 - i. Построить не менее 3-ех разных типов графиков,
 - ii. Проверить не менее 3-ех гипотез,
 - iii. Написать выводы

Оценки

За проект необходимо получить минимум: “Зачет” за базовую часть и **хотя бы 2 балла** за дополнительную часть.

Для дополнительной части:

Максимальный балл за итерацию (0,9 - 1 б):

- Пункт выполнен полностью даже без минорных замечаний (исправлений на месте),
- Студент ответил в полной форме на все дополнительные вопросы (задачи),
- У проверяющего не возникло подозрений на плагиат или не самостоятельное выполнение задания

Средний балл за итерацию (0,5 - 0,9 б):

- Пункт выполнен без критических замечаний (не требует доработки материалов студентом дома), Разрешается иметь 1 пункт с минорными замечаниями
- У проверяющего не возникло подозрений на плагиат или не самостоятельное выполнение задания,
- Студент ответил в полной форме на все дополнительные

вопросы (задачи).

Минимальный балл за итерацию (0,3 - 0,5):

- Большая часть пунктов выполнена без критических замечаний (не требует доработки материалов студентом дома), оставшаяся часть пунктов выполнена с минорными замечаниями
- У проверяющего не возникло подозрений на плагиат или не самостоятельное выполнение задания,
- Студент ответил частично на доп / уточняющие вопросы.

Не засчитана на сдаче (0,6):

- Пункт выполнен с критическими замечаний (требует доработки материалов студентом дома),
- У проверяющего возникло подозрений на плагиат или не самостоятельное выполнение задания,
- Студент не ответил на доп / уточняющие вопросы.

ДЕДЛАЙНЫ

Все пункты должны быть сданы и утверждены до дедлайна. При сдаче в последние дни перед дедлайном или в день дедлайна, если что-то требует доработки, то этот пункт не засчитывается.

Рекомендация (не обязательно, но логично): выполнять и сдавать пункты №1 и №2 вместе, №3 и №4 вместе, №5 и №6 вместе

№	Обязательная часть (итерации 1-7)	Дополнительная часть
Дедлайн	30.03	15.05